Lab02 实验报告：多线程编程

本报告对应 `Lab02.md` 中的两个任务：Part I 数独答案校验器（`sudoku.c`）和 Part II 多线程排序程序（`sort.c`）。按实验要求，报告包括六部分：Linux 提供的线程控制方法、程序设计、实验结果与分析、遇到的问题及解决方法、参考资料、建议与意见。编译和使用方法见 `README.md`。

1. Linux 提供的线程控制方法

1.1 Linux 中线程的实现

- 在 Linux 内核看来，进程和线程都是可调度的任务（task）。线程由 `clone(2)` 系统调用创建：创建时通过 `CLONE_VM`、`CLONE_FILES`、`CLONE_SIGHAND`、`CLONE_THREAD` 等标志，指定新任务与创建者共享地址空间、打开的文件、信号处理方式等资源。
- 应用程序一般不直接调用 `clone`，而是使用 glibc 提供的 POSIX 线程库（Pthreads）。现代 Linux 上的实现是 **NPTL**（Native POSIX Threads Library），自 glibc 2.3.2 起提供，依赖 Linux 2.6 内核的功能。
- NPTL 采用 **1:1 模型**：每个 pthread 线程对应一个内核调度实体，由内核直接调度，因此多个线程可以真正在多个 CPU 上并行。互斥锁、线程等待等同步操作基于 `futex(2)` 系统调用实现。
- 同一进程的所有线程属于同一个线程组，`getpid()` 返回相同的 PID；每个线程在内核中另有自己的线程 ID（可用 `gettid()` 获得）。可以用 `ps -eLf`、`top -H` 或查看 `/proc/<pid>/task/` 观察一个进程中的各个线程。
- **共享与私有**：同一进程的线程共享全局数据和堆、打开的文件描述符、信号处置方式、当前目录等；每个线程有自己的栈（局部变量）、`errno`、信号掩码、调度策略和优先级，以及 CPU 亲和性。

1.2 使用 Pthreads 的基本约定

- 包含头文件 `<pthread.h>`。传统写法是在编译和链接时加 `-pthread`；从 glibc 2.34 起，原来 libpthread 中的功能已并入 libc，不加也能正常链接（本项目的编译命令就没有加，见 3.1 节），但 `-pthread` 仍是可移植性最好的写法。
- **返回值**：大多数 pthread 函数成功时返回 0，失败时直接返回错误码，**不设置** `errno`。所以应该用 `strerror(返回值)` 得到错误说明，而不是用依赖 `errno` 的 `perror`。本实验两个程序中，`pthread_create` 失败时都是这样处理的。

- **线程标识**：`pthread_t` 只在本进程内唯一。比较两个线程标识应使用 `pthread_equal`，不应假设它是整数而直接用 `==` 比较。

1.3 线程控制函数

类别	函数	说明
创建	`pthread_create(&tid, attr, func, arg)`	创建新线程执行 `func(arg)`，线程标识写入 `tid`；`attr` 为 `NULL` 时使用默认属性
终止	线程函数 `return`，或调用 `pthread_exit(retval)`	结束当前线程并给出返回值。注意：任一线程调用 `exit()`，或 `main` 返回，都会结束整个进程
取消	`pthread_cancel(tid)`	请求取消另一个线程；默认在"取消点"（如 `pthread_join`、`sleep`、`read`）生效
等待	`pthread_join(tid, &retval)`	阻塞直到指定线程结束，取得其返回值并回收其资源。可汇合的线程若既不 join 也不 detach，结束后资源不会释放（"僵尸线程"）
分离	`pthread_detach(tid)`	线程结束后由系统自动回收资源，之后不能再 join
标识	`pthread_self()`、`pthread_equal(t1, t2)`	获得当前线程的标识；比较两个标识
属性	`pthread_attr_init`、`pthread_attr_setdetachstate`、`pthread_attr_setstacksize`、`pthread_attr_destroy`	创建线程前设置分离状态、栈大小等
一次性初始化	`pthread_once(&once, init)`	保证 `init` 在多线程环境下只执行一次
线程私有数据	`pthread_key_create`、`pthread_setspecific`、`pthread_getspecific`	同一个键在每个线程中对应各自的一份数据
调度与 CPU 亲和	`sched_yield()`、`pthread_setschedparam`、`pthread_setaffinity_np`（GNU 扩展）；命令行工具 `taskset`	主动让出 CPU；设置调度策略和优先级；限定线程可以运行的 CPU
信号	`pthread_kill(tid, sig)`、`pthread_sigmask`	向指定线程发送信号；设置当前线程的信号掩码

1.4 线程同步方法

机制	主要函数	用途
线程等待	pthread_join	最简单的同步：等一个线程完全结束
互斥锁	pthread_mutex_lock、pthread_mutex_unlock、pthread_mutex_trylock；静态初始化 PTHREAD_MUTEX_INITIALIZER	保证同一时刻只有一个线程进入临界区
条件变量	pthread_cond_wait、pthread_cond_signal、pthread_cond_broadcast；PTHREAD_COND_INITIALIZER	与互斥锁配合使用，让线程等待某个条件成立
读写锁	pthread_rwlock_rdlock、pthread_rwlock_wrlock、pthread_rwlock_unlock	允许多个线程同时读，写时独占
屏障	pthread_barrier_init、pthread_barrier_wait	一组线程都到达同一点后再一起继续
自旋锁	pthread_spin_lock、pthread_spin_unlock	忙等待的轻量级锁，适合临界区极短的场合
信号量	sem_init、sem_wait、sem_post、sem_destroy（<semaphore.h>）	计数信号量，可用于互斥或资源计数

POSIX 标准规定，上表中的 `pthread_create`、`pthread_join`、加锁和解锁、条件变量的等待和通知、屏障等待、信号量操作等函数会**同步内存**：一个线程在调用这些函数之前写入的数据，另一个线程在对应的同步操作之后一定能看到。本实验的两个程序正是依靠 `pthread_create` 和 `pthread_join` 的这一保证，让主线程和工作线程之间正确地传递数据。

1.5 用法示例

以下示例都在 3.1 节的实验环境中编译并运行过。示例 1、3 使用与项目相同的选项；示例 2 需要让两个线程真正交错执行才能看到数据竞争，因此特意关掉优化，用 `gcc -std=c99 -Wall 源文件 -o 程序名` 编译。

示例 1：创建线程、传递参数、取回返回值。

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct
{
    int from;
    int to;
} range;

/* 线程函数：计算 from 到 to 的和，通过返回值把结果交给 pthread_join */
void *sum_range(void *arg)
{
    range *r = (range *) arg;
    long *sum = (long *) malloc(sizeof(long));
    int i;

    *sum = 0;
    for (i = r->from; i <= r->to; i++)
        *sum += i;
    return sum;
}

int main(void)
{
    pthread_t tid;
    range r = {1, 100};
    void *ret;
    int err;

    err = pthread_create(&tid, NULL, sum_range, &r);
    if (err != 0) {
        fprintf(stderr, "pthread_create: %s\n", strerror(err));
        return 1;
    }
    pthread_join(tid, &ret);
    printf("sum = %ld\n", *(long *) ret);
    free(ret);
    return 0;
}
```

运行输出 `sum = 5050`。要点：

- 多个参数装进结构体，把结构体地址作为 `void *` 传给线程。这里 `r` 是 `main` 的局部变量，之所以安全，是因为 `main` 在 `r` 失效之前一直用 `pthread_join` 等着线程。如果创建者可能先于线程使用参数而返回（例如本实验中的 `create_worker` 函数），参数就必须用 `malloc` 申请。

- 线程的返回值必须指向线程结束后仍然有效的内存（这里用 `malloc`），不能指向线程自己的局部变量。

示例 2：用互斥锁保护共享变量。

```c
#include <pthread.h>
#include <stdio.h>

#define TIMES 1000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg)
{
    int i;

    for (i = 0; i < TIMES; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main(void)
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, add, NULL);
    pthread_create(&t2, NULL, add, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %ld\n", counter);
    return 0;
}
```

两个线程各把 `counter` 加 100 万次。各运行 5 次的结果：

版本	5 次运行的 counter
加锁（上面的代码）	2000000、2000000、2000000、2000000、2000000
删掉加锁和解锁两行	1141955、1002417、1096993、1101698、1115073

`counter++` 实际上是"读出、加 1、写回"三步。不加锁时，两个线程的这三步会交错执行，一个线程的写回覆盖了另一个线程的结果，于是丢失了大量更新，而且每次丢失的数量都不同。这就是**数据竞争**。

示例 3：用条件变量等待其他线程完成。 以 Part II 为例，如果想一开始就把三个线程都创建出来，可以让合并线程用条件变量等待两个排序线程完成：

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t sorted_cond = PTHREAD_COND_INITIALIZER;
int finished = 0;            /* 已完成的排序线程数 */

/* 每个排序线程排完后执行： */
pthread_mutex_lock(&lock);
finished++;
pthread_cond_signal(&sorted_cond);
pthread_mutex_unlock(&lock);

/* 合并线程开始合并前执行： */
pthread_mutex_lock(&lock);
while (finished < 2)
    pthread_cond_wait(&sorted_cond, &lock);
pthread_mutex_unlock(&lock);
```

`pthread_cond_wait` 会原子地释放互斥锁并进入等待，被唤醒后重新加锁再返回。等待条件要写在 `while` 循环里，因为线程可能在条件尚未满足时被唤醒（虚假唤醒）。把这段代码放进一个先创建合并线程、再创建两个排序线程的测试程序，在 ThreadSanitizer 下运行 50 次，合并线程每次都在两个排序线程完成之后才继续，没有报告数据竞争。本实验的 `sort.c` 采用了更简单的做法（见 2.3 节），没有用到条件变量。

1.6 本实验用到的方法

方法	用途
`pthread_create`	创建工作线程，并把 `parameters` 结构体的地址传给它
`pthread_join`	Part I：等 11 个工作线程全部结束。Part II：先等两个排序线程，再创建合并线程，最后等合并线程
全局数据 + 互不重叠的写入区域	代替锁：每个线程只写属于自己的位置，因此两个程序都不需要互斥锁
`taskset`	性能实验中让程序只在一个 CPU 上运行，作为对比的基准（见 3.3 节）

2. 程序设计

2.1 两个程序共同的设计

- **输入输出**：两个程序都从标准输入读数据，可以用 `<` 重定向文件或用管道 `|` 输入；结果写到标准输出，错误信息写到标准错误。输入不合法时打印错误信息并以退出码 1 结束。

- **线程参数**：沿用实验文档的建议，把线程需要的位置信息放进 `parameters` 结构体。辅助函数 `create_worker` 负责 `malloc` 一个结构体、填写参数、调用 `pthread_create`；工作线程读完参数后自己 `free`。这样每个线程拿到的都是自己独立的一份参数，且在线程使用期间一直有效。
- **共享数据与同步**：线程之间通过全局数组共享数据。设计时保证任意两个可能同时运行的线程不会写同一个位置，再用 `pthread_join` 安排先后顺序，因此不需要锁。
- **错误处理**：`malloc`、`realloc` 失败时用 `perror` 打印原因；`pthread_create` 失败时用 `strerror(err)` 打印其返回的错误码（见 1.2 节）。

2.2 Part I：数独答案校验器

线程分工（实验文档建议的 11 线程方案）：

线程	执行的函数	参数 (row, column)	检查的区域	结果写入
0	`check_rows`	(0, 0)	全部 9 行	`result[0]`
1	`check_columns`	(0, 0)	全部 9 列	`result[1]`
2 + i（i = 0～8）	`check_subgrid`	((i / 3) × 3, (i % 3) × 3)	第 i + 1 个 3×3 子格（从左到右、从上到下）	`result[2 + i]`

数据结构：

- `int sudoku[9][9]`：全局数组，存放数独。主线程在创建工作线程之前读入，之后只读。
- `int result[11]`：全局结果数组，`result[i]` 由线程 i 写入，1 表示合格，0 表示不合格。
- `parameters { int row; int column; }`：传给线程的起始行和起始列，与实验文档给出的结构一致。

结果下标：为了不改动实验文档给出的 `parameters` 结构，子格线程不额外接收编号，而是由起点算出自己在 `result` 中的下标：

```
int index = 2 + (data->row / 3) * 3 + data->column / 3;
```

例如起点为 (3, 6) 的子格，下标为 2 + 3 + 2 = 7。

检查算法：三种区域都是矩形，所以由同一个函数 `region_is_valid(row, column, height, width)` 检查：行是 1×9，列是 9×1，子格是 3×3。函数用长度为 10 的数组 `seen` 记录 1～9 各数字是否出现过，遇到不在 1～9 之间的数或重复的数就返回 0。区域中恰好有 9 个数，它们都在 1～9 之间且互不重复，就说明恰好包含 1～9 各一次。

运行流程：

```
main thread                          worker threads
──────────                           ──────────────
1. read 81 numbers into sudoku

2. create worker 0   ──────────→    worker 0 : check all rows      → result[0]
   create worker 1   ──────────→    worker 1 : check all columns   → result[1]
   create worker 2   ──────────→    worker 2 : check one subgrid   → result[2]
   ...                               ...
   create worker 10  ──────────→    worker 10: check one subgrid   → result[10]

3. join worker 0 .. 10               (the 11 workers run at the same time,
   (wait until all have finished)     in an order decided by the OS)

4. read result[0..10], print the verdict
```

主线程依次创建 11 个线程，不等待任何一个；11 个线程互不依赖，可以同时运行。主线程用 `pthread_join` 等全部线程结束后，按编号 0～10 打印每个线程的结果，只有 11 个结果全为 1 时才判定数独正确。

为什么不需要锁：`sudoku` 在创建线程前写好，之后所有线程只读；每个线程只写 `result` 中自己的一格；主线程在 `pthread_join` 返回之后才读 `result`。

2.3 Part II：多线程排序

线程分工：

线程	执行的函数	参数 `[start, end)`	工作
0	`sorter`	`[0, middle)`	用插入排序把前一半原地排好
1	`sorter`	`[middle, count)`	用插入排序把后一半原地排好
2	`merger`	`[0, count)`	把排好的两半合并进 `sorted`

其中 `middle = count / 2`。个数为奇数时后一半多 1 个；只有 1 个数时前一半为空，线程 0 仍会被创建，只是没有数可排。

数据结构：

- `int *list`：全局指针，指向读入的列表。输入长度事先未知，所以读入时用 `realloc` 按需扩大，容量从 16 开始，每次放满就翻倍。
- `int *sorted`：全局指针，指向与 `list` 同样大小（`count` 个 `int`）的第二个数组，用来存放合并结果，对应实验文档要求的"第二个全局数组"。
- `int count`：元素个数。
- `parameters { int start; int end; }`：线程负责的左闭右开范围，包含 `start`、不包含 `end`。这样前一半的 `end` 正好等于后一半的 `start`，两段既不重叠也不遗漏。

算法：

- 排序线程用**插入排序**，原地排序，时间复杂度为 $O(m^2)$（m 为这一半的长度）。内层循环条件 `j ≥ data→start` 保证线程只访问自己的范围。
- 合并线程用三个下标 `i`、`j`、`k` 分别指向前一半、后一半和 `sorted` 的当前位置，每次取两半当前元素中较小的一个（相等时取前一半的）放入 `sorted`；一半用完后，把另一半剩下的元素依次抄过去。时间复杂度为 O(n)。合并线程用 `(start + end) / 2` 求出中点，与主线程的 `count / 2` 相同，因此找到的分界正好是两个排序线程的分界。

线程协调：合并线程依赖两个排序线程的结果，必须在它们完成后才能开始。本程序让主线程先 `pthread_join` 两个排序线程，再创建合并线程：

```
    /* split the list in two; if count is odd, the second half is one longer */
    middle = count / 2;
    create_worker(&workers[0], sorter, 0, middle);
    create_worker(&workers[1], sorter, middle, count);
    pthread_join(workers[0], NULL);
    pthread_join(workers[1], NULL);

    /* the merging thread may start only after both halves are sorted */
    create_worker(&workers[2], merger, 0, count);
    pthread_join(workers[2], NULL);
```

运行流程：

```
main thread                          worker threads
──────────                           ──────────────
1. read the integers into list
2. allocate sorted, print the original list

3. create worker 0  ──────────────→   worker 0: sort the first half of list
   create worker 1  ──────────────→   worker 1: sort the second half of list
   join worker 0                     (workers 0 and 1 run at the same time)
   join worker 1

4. create worker 2  ──────────────→   worker 2: merge both halves into sorted
   join worker 2

5. print the results, free the memory
```

程序分为两个阶段：先是两个排序线程并行工作，然后合并线程单独工作。原始列表在创建线程之前打印（因为排序线程会原地修改 `list`）；两个排好的半段和最终结果在全部线程结束之后打印，满足"所有线程结束后由父线程输出"的要求。

为什么不需要锁：两个排序线程同时运行，但只访问 `list` 中互不重叠的两段；合并线程运行时两个排序线程已经结束，它只读 `list`、只写 `sorted`；主线程在 `pthread_join` 返回之后才读取结果。

3. 实验结果与分析

3.1 实验环境

项目	配置
操作系统	Ubuntu 26.04.1 LTS，运行在 WSL2 中
内核	Linux 6.18.54.1-microsoft-standard-WSL2（x86_64）
CPU	Intel Core i9-14900HX，系统中可见 32 个逻辑 CPU
内存	15 GiB
编译器	GCC 15.2.0
C 库	glibc 2.43（NPTL）

构建命令如下，可执行文件生成在 `build/` 目录中（该目录已写入 `.gitignore`）：

```
mkdir -p build && gcc -std=c99 -O2 -Wall -fno-asm "sudoku.c" -o "build/sudoku" --static -lm
mkdir -p build && gcc -std=c99 -O2 -Wall -fno-asm "sort.c" -o "build/sort" --static -lm
```

两个程序用上面的命令编译均无警告；另外加上 `-Wextra -pedantic` 检查也没有警告。`-std=c99` 按 C99 标准编译，`-O2` 开启二级优化，`-fno-asm` 禁用 `asm` 关键字（程序没有使用内联汇编），`--static` 静态链接，`-lm` 链接数学库。

3.2 Part I：数独答案校验器

测试用例：以实验文档中的数独为基础，构造了以下输入：

输入文件	说明	报告不合格的线程	结论	退出码
`valid.txt`	实验文档中的数独	无	valid	0
`swap_same_box.txt`	交换第 1 行第 1、2 列的 6 和 2（两格在同一子格内）	1	invalid	0
`swap_cross_box.txt`	交换第 1 行第 3、4 列的 4 和 5（两格在相邻的两个子格中）	1、2、3	invalid	0
`dup_center.txt`	把第 5 行第 5 列的 4 改为 5	0、1、6	invalid	0
`latin.txt`	第 i 行为 1～9 循环左移 i − 1 位	2～10	invalid	0
`zero.txt`	把第 9 行第 9 列的 6 改为 0	0、1、10	invalid	0
`short.txt`	只有 80 个整数	—	报错	1
`letter.txt`	把第一个数换成字母 `x`	—	报错	1

运行结果截图（终端中的实际输出）：

```
$ mkdir -p build && gcc -std=c99 -O2 -Wall -fno-asm "sudoku.c" -o "build/sudoku" --static -lm
$ ./build/sudoku < valid.txt
Thread  0: valid    all rows
Thread  1: valid    all columns
Thread  2: valid    subgrid rows 1-3, columns 1-3
Thread  3: valid    subgrid rows 1-3, columns 4-6
Thread  4: valid    subgrid rows 1-3, columns 7-9
Thread  5: valid    subgrid rows 4-6, columns 1-3
Thread  6: valid    subgrid rows 4-6, columns 4-6
Thread  7: valid    subgrid rows 4-6, columns 7-9
Thread  8: valid    subgrid rows 7-9, columns 1-3
Thread  9: valid    subgrid rows 7-9, columns 4-6
Thread 10: valid    subgrid rows 7-9, columns 7-9
The Sudoku solution is valid.
$ ./build/sudoku < dup_center.txt
Thread  0: invalid  all rows
Thread  1: invalid  all columns
Thread  2: valid    subgrid rows 1-3, columns 1-3
Thread  3: valid    subgrid rows 1-3, columns 4-6
Thread  4: valid    subgrid rows 1-3, columns 7-9
Thread  5: valid    subgrid rows 4-6, columns 1-3
Thread  6: invalid  subgrid rows 4-6, columns 4-6
Thread  7: valid    subgrid rows 4-6, columns 7-9
Thread  8: valid    subgrid rows 7-9, columns 1-3
Thread  9: valid    subgrid rows 7-9, columns 4-6
Thread 10: valid    subgrid rows 7-9, columns 7-9
The Sudoku solution is invalid.
$ ./build/sudoku < short.txt
Error: expected 81 integers on standard input
$ echo $?
1
```

结果分析：

- `swap_same_box`：交换的两个数仍在同一行、同一子格中，所以行和子格都合格；但第 1、2 列各自出现了重复的数，只有检查列的线程 1 报错。
- `swap_cross_box`：第 3、4 列出现重复，线程 1 报错；第 1 个子格换进来的 5 与第 2 行第 1 列的 5 重复，第 2 个子格换进来的 4 与第 3 行第 6 列的 4 重复，所以线程 2、3 也报错。行仍然合格。
- `dup_center`：被改的格子同时属于第 5 行、第 5 列和正中间的子格，这三个区域原本已有一个 5，因此线程 0、1、6 报错。
- `latin`：每行、每列都是 1～9 的一个排列，线程 0、1 判定合格；但每个子格都有重复（例如第 1 个子格为 1 2 3 / 2 3 4 / 3 4 5），9 个子格线程全部报错。这个用例说明子格检查是必不可少的。
- `zero`：0 不在 1～9 之间，所在的第 9 行、第 9 列和第 9 个子格（线程 10）都不合格。
- 输入不足 81 个整数或含有非整数时，程序报错并以退出码 1 结束，不会对不完整的数据做检查。

每个用例中报错的线程都恰好是包含被修改格子的区域，与预期完全一致。

重复运行：线程的调度顺序每次都可能不同，为检查结果是否受影响，把 `valid.txt`、`dup_center.txt`、`latin.txt` 各运行 1000 次，每个输入的 1000 次输出都完全相同（只出现 1 种输出）。原因是每个线程的检查结果只取决于数独本身，而打印发生在所有线程结束之后，由主线程按固定顺序进行。

并发正确性检查：

工具	运行次数	报告的问题
ThreadSanitizer（`-fsanitize=thread`，检测数据竞争）	上表 8 个输入各 20 次，共 160 次	0
AddressSanitizer + UBSan（`-fsanitize=address,undefined`，检测内存错误和未定义行为）	8 个输入各 1 次	0

3.3 Part II：多线程排序

运行结果截图（终端中的实际输出）：

```
$ mkdir -p build && gcc -std=c99 -O2 -Wall -fno-asm "sort.c" -o "build/sort" --static -lm
$ echo 7 12 19 3 18 4 2 6 15 8 | ./build/sort
Original list:    7 12 19 3 18 4 2 6 15 8
Sorting thread 0: 3 7 12 18 19
Sorting thread 1: 2 4 6 8 15
Sorted list:      2 3 4 6 7 8 12 15 18 19
$ echo 5 3 1 4 2 | ./build/sort
Original list:    5 3 1 4 2
Sorting thread 0: 3 5
Sorting thread 1: 1 2 4
Sorted list:      1 2 3 4 5
$ echo 42 | ./build/sort
Original list:    42
Sorting thread 0:
Sorting thread 1: 42
Sorted list:      42
$ echo 7, 12, 19 | ./build/sort
Error: expected integers separated by whitespace on standard input
$ echo $?
1
```

第一组是实验文档中的例子，结果与文档 Fig. 2 完全一致。第二组有 5 个数，`middle = 2`，后一半比前一半多 1 个。第三组只有 1 个数，前一半为空。第四组按文档图中的写法用逗号分隔，程序报错并以退出码 1 结束。

正确性统计：

测试	内容	结果
典型输入	上面的例子，以及含负数和重复值的 `3 -1 3 0 -1 2`	全部正确
随机列表	500 个随机列表，长度 1～60，取值 −50～50；逐一检查原始列表、两个半段和最终结果	0 个错误
大规模随机列表	性能实验中使用的 2.5 万～40 万个数（见下文）	结果与 Python `sorted` 完全一致
重复运行	实验文档的例子运行 1000 次	只出现 1 种输出
错误输入	逗号分隔、空输入、含字母	均报错，退出码 1
ThreadSanitizer	6 种输入（含 5000 个随机数）各 20 次，共 120 次	0 个报告
AddressSanitizer + UBSan	6 种输入各 1 次	0 个报告

性能实验：为了观察多线程的效果，用不同规模的随机数据测量运行时间。

- 数据：`shuf -i 1-10000000 -n N` 生成的 N 个互不相同的随机整数。
- 两种运行方式：(a) 不限制 CPU；(b) 用 `taskset -c 0 ./build/sort` 限定只在 0 号 CPU 上运行，此时两个排序线程只能轮流执行，相当于没有并行的基准。
- 每种规模下两种方式交替各运行 5 次（N = 10 时各运行 200 次），输出重定向到 `/dev/null`。`real` 是进程从启动到结束的实际时间，`user` 是所有线程在用户态消耗的 CPU 时间之和。
- 加速比 = 方式 (b) 的平均 `real` ÷ 方式 (a) 的平均 `real`。

N	(a) 不限制 CPU：`real`（均值 ± 标准差，秒）	(a) `user`（秒）	(b) 只用 1 个 CPU：`real`（秒）	(b) `user`（秒）	加速比
10	0.0006 ± 0.0001	0.0003	0.0007 ± 0.0001	0.0005	1.31
25,000	0.0136 ± 0.0005	0.0189	0.0233 ± 0.0005	0.0216	1.71
50,000	0.0455 ± 0.0005	0.0786	0.0836 ± 0.0017	0.0795	1.84
100,000	0.1699 ± 0.0082	0.3163	0.3193 ± 0.0038	0.3171	1.88
200,000	0.6556 ± 0.0054	1.2770	1.2584 ± 0.0054	1.2564	1.92
400,000	2.5833 ± 0.0114	5.1024	5.0401 ± 0.0382	5.0348	1.95

另外，N = 1,000,000 时以方式 (a) 运行 1 次，`real` 为 15.6 秒。

规模每翻一倍，平均 `real` 的增长倍数：

N 的变化	(a) 不限制 CPU	(b) 只用 1 个 CPU
25,000 → 50,000	3.35	3.59
50,000 → 100,000	3.73	3.82
100,000 → 200,000	3.86	3.94
200,000 → 400,000	3.94	4.01

分析：

1. **两个排序线程确实在并行工作**。 N 不小于 25,000 时，加速比为 1.71～1.95，接近两个线程的理论上限 2，并随 N 增大逐渐提高。不限制 CPU 时，`user` 约为 `real` 的 2 倍，说明有两个 CPU 在同时执行程序；只用 1 个 CPU 时，`user` 与 `real` 基本相等。N 较大时两种方式的 `user` 几乎相同，说明并行并没有减少总工作量，只是把工作分给了两个 CPU 同时完成。
2. **加速比达不到 2 的原因**。 根据 Amdahl 定律，只有可并行部分能被加速：$S = 1 / ((1 - p) + p / 2)$，其中 p 为可并行部分所占的比例。程序中进程启动、读入数据、申请内存、创建线程、合并（O(n)）和输出都是串行的。由 S = 1.95 反推，p 约为 0.97。N 较小时串行部分占比更大：N = 25,000 时排序本身只有十几毫秒，进程启动、读入数据和创建线程等固定开销占比更高，加速比只有 1.71；N 增大后，串行部分的占比下降，加速比逐渐接近 2。WSL2 的虚拟 CPU 调度和 CPU 频率变化也会带来少量波动。
3. **运行时间随规模平方增长**。 N 每翻一倍，用时增长约 3.4～4 倍，符合插入排序 $O(n^2)$ 的复杂度；合并只需 O(n) 时间，在总用时中可以忽略。按这个规律，由 N = 400,000 的 2.58 秒推算 N = 1,000,000 的用时约为 $2.58 \times 2.5^2 \approx 16.1$ 秒，与实测的 15.6 秒基本一致。
4. **"分成两半"本身也减少了工作量**。 插入排序的工作量约与长度的平方成正比。分别排序两个长度为 n/2 的半段，总工作量约为 $2 \times (n/2)^2 = n^2/2$，只有直接对整个列表做插入排序的一半，而合并只增加 O(n) 的工作。因此，即使只有 1 个 CPU，本程序也应比单线程地对整个列表做插入排序快约一倍；在多核上并行后，用时约为后者的四分之一。这一点是理论分析，本实验没有另外编写单线程版本做对照测量。
5. **数据很少时，多线程没有收益**。 N = 10 时，一次运行只需约 0.6 毫秒，几乎全部是进程启动和创建线程的开销，排序 10 个数的时间可以忽略。此时方式 (b) 略慢，主要是因为 `taskset` 本身也是一个需要启动的程序，它设置好 CPU 亲和性后再启动 `build/sort`，增加了一次程序启动的开销，与并行无关。Part I 的情况与此类似：每个线程只检查 9～81 个数，创建线程的开销远大于检查本身，使用多线程的意义在于练习线程的创建、参数传递和结果收集，而不是提速。

4. 遇到的问题及解决方法

问题 1：线程函数只能接收一个 `void *` 参数，但每个线程需要知道多个值（起始行和列，或起点和终点）。 解决：按实验文档的建议，把这些值放进结构体 `parameters`，把结构体的地址作为参数传给 `pthread_create`，线程函数中再把 `void *` 转换回 `parameters *` 来读取。

问题 2：参数结构体的生命周期。 创建线程的工作封装在辅助函数 `create_worker` 中。如果在其中用局部变量存放参数并传递它的地址，函数一返回这块内存就失效，下一次调用还可能复用同一块内存；而新线程什么时候开始读取参数是不确定的，可能读到错误的数据。 解决：在 `create_worker` 中用 `malloc` 为每个线程单独申请参数结构体，由线程用完后自己 `free`。

问题 3：pthread 函数出错时如何报告。 `pthread_create` 失败时不设置 `errno`，而是直接返回错误码，用 `perror` 会打印出与本次错误无关的信息。 解决：保存返回值 `err`，用 `strerror(err)` 打印错误说明（见 1.2 节）。

问题 4（Part I）：子格线程如何知道应把结果写进 `result` 的哪一格。 实验文档给出的 `parameters` 只有 `row` 和 `column` 两个成员。 解决：不修改结构体，由子格起点计算下标 `2 + (row / 3) * 3 + column / 3`，9 个子格正好对应 2～10。

问题 5（Part I）：多个线程同时写结果数组是否需要加锁。 解决：让线程 i 只写 `result[i]`，各线程写的位置互不相同，不存在数据竞争；主线程用 `pthread_join` 等所有线程结束后再读取。用 ThreadSanitizer 运行 160 次，没有报告数据竞争。

问题 6：输入不完整或格式错误。 解决：检查 `scanf` 的返回值。Part I 必须读满 81 个整数；Part II 在输入结束前遇到非整数，或一个整数都没有读到，都视为错误。两种情况都向标准错误打印提示，并以退出码 1 结束，不对不完整的数据做处理。

问题 7（Part II）：合并线程必须等两个排序线程都完成。 如果三个线程同时启动，合并线程可能读到尚未排好、甚至正在被修改的数据，而且结果会随调度顺序变化，时对时错。 解决：主线程先用 `pthread_join` 等两个排序线程结束，再创建合并线程。合并线程在两半排好之前根本不存在，顺序得到保证。1.5 节的示例 3 给出了用条件变量实现的另一种做法。

问题 8（Part II）：输入的整数个数事先未知。 解决：`list` 定义为全局指针，读入时用 `realloc` 动态扩大，容量从 16 开始，每次放满就翻倍，扩大的次数只与 log n 成正比。读完后再按实际个数 `malloc` 第二个数组 `sorted`。

问题 9（Part II）：个数为奇数时无法"平分"，合并线程也需要知道两半的分界。 解决：取 `middle = count / 2`，后一半多 1 个数。合并线程收到整个范围 `[0, count)`，用同样的方法求中点，得到的分界与主线程给排序线程的分界一致。只有 1 个数时前一半为空，程序同样正确。

问题 10（Part II）：两个排序线程同时修改同一个数组，如何保证互不干扰。 解决：用左闭右开范围 `[start, end)` 描述每个线程负责的一段，两段首尾相接、不重叠。插入排序的内层循环条件写成 `j ≥`

`data->start && list[j] > value`：利用 `&&` 的短路求值，`j` 一旦越过范围左端就不再读 `list[j]`，线程 1 因而不会去读线程 0 正在修改的那一半。

问题 11（Part II）：合并结果不能直接写回 `list`。 原地合并会覆盖尚未参与合并的元素，例如合并 3 7 和 2 4 时，第一步把 2 写到下标 0，就会覆盖还没用到的 3。解决：按实验文档的设计，把合并结果写进第二个全局数组 `sorted`。

问题 12（Part II）：实验文档图中的列表用逗号分隔，与程序的输入格式不同。 解决：程序只接受用空格或换行分隔的整数。遇到逗号时给出明确的错误提示 `Error: expected integers separated by whitespace on standard input`，并在 `README.md` 中说明输入格式。

问题 13（Part II）：可执行文件名 `sort` 与系统自带的 `sort` 命令同名。 只输入 `sort` 时，shell 会在 `PATH` 中找到系统命令，它按文本逐行排序，结果与本程序不同。解决：把可执行文件放在 `build/` 目录中，运行时写 `./build/sort`，用路径明确指定要运行的程序，并在 `README.md` 中注明。

问题 14（实验）：如何证明加速来自并行，并减少测量误差。 解决：用同一个程序做对比，以 `taskset -c 0` 限定只用 1 个 CPU 的运行作为基准，排除了"分成两半"本身减少工作量带来的影响（两种方式下的工作量相同）。每种规模交替运行 5 次，取平均值并给出标准差；同时记录 `user` 时间，从 CPU 时间的角度确认两个 CPU 在同时工作。

问题 15（实验）：插入排序处理大规模数据很慢。 100 万个数需要约 16 秒。解决：本实验的重点是线程，插入排序代码简单，并且原地排序正好符合"每个排序线程处理数组的一半"的设计，因此保留了它，并在报告中分析了它的复杂度。如果需要处理更大的数据，只需在 `sorter` 中改用归并排序、快速排序或 C 标准库的 `qsort` 等 O(n log n) 算法，线程的创建、传参和合并都不用改。

5. 参考资料

1. `Lab02.md`：本实验的实验指导。
2. A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Wiley, 2018. 第 4 章 Threads & Concurrency，其中的编程项目 Sudoku Solution Validator 和 Multithreaded Sorting Application 即本实验的两个任务。
3. M. Kerrisk. *The Linux Programming Interface*. No Starch Press, 2010. 第 29～33 章（线程）。
4. Linux man-pages：pthreads(7)，https://man7.org/linux/man-pages/man7/pthreads.7.html
5. Linux man-pages：nptl(7)，https://man7.org/linux/man-pages/man7/nptl.7.html
6. Linux man-pages：pthread_create(3)，https://man7.org/linux/man-pages/man3/pthread_create.3.html；pthread_join(3)，https://man7.org/linux/man-pages/man3/pthread_join.3.html
7. Linux man-pages：clone(2)，https://man7.org/linux/man-pages/man2/clone.2.html
8. Linux man-pages：taskset(1)，https://man7.org/linux/man-pages/man1/taskset.1.html

9. IEEE Std 1003.1-2017（POSIX.1-2017），Base Definitions 第 4.12 节 Memory Synchronization，https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap04.html
10. The GNU C Library 2.34 发布公告（libpthread 并入 libc），https://sourceware.org/pipermail/libc-alpha/2021-August/129718.html
11. GCC 手册：Program Instrumentation Options（`-fsanitize=thread`、`-fsanitize=address`、`-fsanitize=undefined`），https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html
12. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. 第 2 章（插入排序与归并）。

6. 建议与意见

1. **明确输入格式并提供测试数据。** Part II 的图中列表用逗号分隔，但文档没有规定程序的输入格式。建议明确输入格式，并提供若干组测试输入和预期输出，方便自查。
2. **说明奇数长度的处理方式。** Part II 要求把列表分成"大小相等"的两个子列表，但长度为奇数时无法做到。建议说明这种情况下哪一半多一个元素，或说明可以自行决定。
3. **把线程同步作为可选的扩展要求。** Part II 只要求父线程在所有线程结束后输出，没有规定合并线程何时启动、如何等待排序线程。最简单的 `pthread_join` 方案就能满足要求。建议增加一个可选要求，例如三个线程同时创建，由合并线程用条件变量或屏障等待排序线程，以便练习互斥锁和条件变量等同步工具。
4. **说明"统计数据"的范围。** 报告要求给出实验结果的统计数据，但实验本身没有规定要测量什么。建议给出参考方向，例如比较不同数据规模，或比较单核和多核下的运行时间。
5. **小的文字问题。** Part II 中 "Refer to the instructions in Project 1" 应为 Part I；Part I 所说的 "the following figure" 实际是一个表格；文档中只有 Fig. 2，没有 Fig. 1。
6. **体会。** Part I 的各线程互不依赖，只要让每个线程写不同的位置，就完全不需要同步；Part II 的合并线程依赖排序线程，必须明确安排先后顺序。两部分都说明，合理划分数据，让线程尽量不共享可写的数据，是写出正确多线程程序最简单的办法。另外，数据竞争造成的错误不一定每次都出现（1.5 节示例 2 中，不加锁时 5 次运行得到 5 个不同的结果），ThreadSanitizer 这类工具对发现这类问题很有帮助。